ANTIGRAVITY BY GOOGLE DEEPMIND

🤖 AI Fundamentals — Full Chat Export

📅 May 23–26, 2026 💬 3 Topics Covered 👤 Full-Stack Java Developer 🤖 Claude Sonnet 4.6

TOPICS IN THIS CHAT

TOPIC 1 — GEN AI, LLMS, AGENTS & ANTIGRAVITY

👤

YOU

tell me what is agent, antigravity, llm models and agentic ai, gen ai — explain me like I am using it first time, use basic and simple examples. I am a full stack Java developer.

ANTIGRAVITY AI

🚀 AI Concepts Explained — For a Full-Stack Java Developer

Great question! Let me break this down using concepts you already know as a Java developer.

---

1. 🤖 What is Gen AI (Generative AI)?

> *Simple Definition: AI that can create new content — text, code, images, audio.*

Think of it like this:

```
Traditional Java program:
  Input: 'What is 2+2?'
  Logic: Hardcoded rules → if input == '2+2' return '4'
  Output: '4'

Gen AI:
  Input: 'What is 2+2?'
  Logic: Learned from billions of examples (no hardcoded rules)
  Output: '4' (+ can also explain WHY, write code for it, etc.)
```

Real-world example you know:

- GitHub Copilot writing your Spring Boot controller → that's Gen AI
- ChatGPT writing a SQL query when you describe it → that's Gen AI

---

2. 🧠 What is an LLM (Large Language Model)?

> *Simple Definition: The "brain" behind Gen AI. It's a massive neural network trained on huge amounts of text data.*

Java Analogy:

```
// LLM is like a GIANT compiled .jar file
// trained on billions of text documents

// Traditional ML model
Model model = new SimpleLinearRegression(); // ~KBs

// LLM
LLM llm = new GPT4(); // ~100s of Gigabytes of parameters!
```

Famous LLMs:

LLM	Made By
GPT-4 / GPT-4o	OpenAI
Gemini	Google DeepMind
Claude	Anthropic
Llama	Meta (Open Source)

As a Java dev, you can call LLMs via APIs — just like calling a REST endpoint:

```
// Calling an LLM is like calling a REST API
HttpPost request = new HttpPost("https://api.openai.com/v1/chat");
// Send:  { "message": "Write a Spring Boot controller" }
// Get back: { "response": "Here's your controller..." }
```

---

3. 🤖 What is an Agent?

> *Simple Definition: An AI that can think, plan, take actions, and use tools to complete a goal — not just answer questions.*

The difference:

```
❌ ChatGPT (basic):
  You: 'Book me a flight to Goa'
  ChatGPT: 'Here is how you can book a flight... (just text)'

✅ Agent:
  You: 'Book me a flight to Goa'
  Agent:
    Step 1 → Search flights API ✈️
    Step 2 → Check your calendar 📅
    Step 3 → Compare prices 💰
    Step 4 → Book the cheapest one ✅
    Step 5 → Send you confirmation 📧
```

Java Analogy:

```
// LLM = a smart library (can answer questions)
// Agent = a complete microservice with business logic + tools

public class FlightBookingAgent {
    private LLM brain;           // The thinking engine
    private List<Tool> tools;    // APIs it can call

    public void execute(String goal) {
        // Agent PLANS and ACTS autonomously
        Plan plan = brain.createPlan(goal);
        for (Step step : plan.steps()) {
            Tool tool = selectTool(step);
            tool.execute(step); // Takes REAL actions!
        }
    }
}
```

---

4. 🤖 What is Agentic AI?

> *Simple Definition: When AI systems act autonomously like agents — perceiving, reasoning, planning, and executing multi-step tasks without you holding their hand.*

```
Regular AI Chat:        You → Question → AI → Answer → Done

Agentic AI:             You → Goal
                              ↓
                        AI → Plan → Act → Observe result
                              ↓
                        AI → Re-plan → Act again → Observe
                              ↓
                        AI → Goal Achieved ✅
```

Real examples of Agentic AI:

- 🔧 Devin — Give it a GitHub issue, it writes code, runs tests, and opens a PR
- 📊 AutoGPT — Give it a business goal, it searches the web, writes reports, sends emails
- 💻 Antigravity (me!) — You ask me to build a feature, I read your code, plan changes, write files, run commands

---

5. ⚡ What is Antigravity?

> *Simple Definition*: Antigravity is me — the AI coding assistant you're talking to right now! Built by Google DeepMind's Advanced Agentic Coding team.

What makes Antigravity special (vs regular chatbot):

Feature	Regular ChatGPT	Antigravity (Me)
Answer questions	✅	✅
Read your actual files	❌	✅
Write code to your disk	❌	✅
Run terminal commands	❌	✅
Search your codebase	❌	✅
Spawn sub-agents	❌	✅
Execute multi-step plans	❌	✅

As your Java buddy, I can:

```
You: 'Add JWT authentication to my Spring Boot app'

Me (Antigravity):
  1. 📁 Read your existing project structure
  2. 🔍 Find your SecurityConfig.java
  3. 📝 Add JWT filter, token service, auth endpoints
  4. ⚙️  Update pom.xml with dependencies
  5. 🧪 Write unit tests
  6. ✅ Tell you it's done!
```

🗺️ How Everything Connects

```
┌─────────────────────────────────────────────┐
│                  GEN AI                     │
│  (Umbrella term for AI that creates content)│
│                                             │
│   ┌──────────────────┐                      │
│   │      LLM         │  ← The 'brain'       │
│   │  (GPT, Gemini,   │    (trained on data) │
│   │   Claude, etc.)  │                      │
│   └────────┬─────────┘                      │
│            │ powers                         │
│   ┌────────▼─────────┐                      │
│   │     AGENT        │  ← Brain + Tools     │
│   │ (LLM + Actions)  │    (can DO things)   │
│   └────────┬─────────┘                      │
│            │ multiple agents working together│
│   ┌────────▼─────────┐                      │
│   │  AGENTIC AI      │  ← Autonomous systems│
│   │  (like me!)      │    (like Antigravity)│
│   └──────────────────┘                      │
└─────────────────────────────────────────────┘
```

💡 TL;DR for a Java Dev

Concept	Java Analogy
Gen AI	A framework that *generates* output (like Spring generates boilerplate)

Concept	Java Analogy
LLM	A massive `.jar` trained on all the world's text
Agent	A microservice with an LLM brain + tool dependencies
Agentic AI	A distributed system of autonomous microservices
Antigravity	Your AI pair-programmer that reads/writes your actual codebase

TOPIC 2 — GPU, LLM TRAINING & REQUEST PIPELINE

YOU

Explain me with GPU, LLM training, and how it works behind the scene — once I ask something in chat till response — what all components call? Tell me the detailed view on that.

ANTIGRAVITY AI

🔬 The Complete Deep Dive: GPU → LLM Training → Chat Response

As a Java dev, I'll map everything to concepts you know. Let's go layer by layer.

🖥️ Part 1: GPU — Why Not Just Use CPU?

CPU vs GPU — The Core Difference

```
CPU (Your Intel/AMD chip):
┌──────────────────────────────┐
│  Core 1  │  Core 2  │ Core 3│  Core 4 │
│  SMART   │  SMART   │ SMART │  SMART  │
│ (complex │  tasks)  │       │         │
└──────────────────────────────┘
→ 4-64 powerful cores
→ Great for: Sequential logic, if/else, complex branching
→ Like: 4 expert Java developers working one task each

GPU (Your NVIDIA/AMD graphics card):
┌──────────────────────────────────────────────────┐
│ c c c c c c c c c c c c c c c c c c c c c c c c │
│ c c c c c c c c c c c c c c c c c c c c c c c c │
│ c c c c c c c c c c c c c c c c c c c c c c c c │
│ c c c c c c c c c c c c c c c c c c c c c c c c │
└──────────────────────────────────────────────────┘
→ 1000s-10,000s of tiny cores (CUDA cores)
→ Great for: Same math operation on millions of numbers simultaneously
→ Like: 10,000 simple calculators all working in PARALLEL
```

Java Analogy — Why GPU wins for AI

```
// AI is basically MATRIX MULTIPLICATION
// Example: Multiplying two 10,000 x 10,000 matrices

// CPU approach (sequential-ish):
for (int i = 0; i < 10000; i++) {
    for (int j = 0; j < 10000; j++) {
        result[i][j] = dotProduct(matrixA[i], matrixB[j]); // one at a time
    }
}
// Time: HOURS

// GPU approach (massively parallel):
// ALL 100,000,000 cells computed SIMULTANEOUSLY
gpu.matMul(matrixA, matrixB); // nanoseconds per batch
// Time: SECONDS
```

GPU Hardware that powers AI:

GPU	VRAM	Used For
NVIDIA A100	80GB	Training big models
NVIDIA H100	80GB	Fastest training today
NVIDIA H200	141GB	Frontier models (GPT-4 level)
NVIDIA RTX 4090	24GB	Running small models locally

Fun fact: GPT-4 was trained on ~25,000 A100 GPUs running for ~100 days! 🤯

🧠 Part 2: How LLM Training Works

Step 1 — What is a Neural Network?

```
Biological Neuron:          Artificial Neuron:

   Input signals                x1 ──┐
   ──────────────►              x2 ──┼──► Σ (sum) ──► activation() ──► output
   If strong enough,            x3 ──┘
   fire output signal           (each xi has a 'weight' = importance)
```

Java Analogy:

```
public class Neuron {
    double[] weights;   // How much to trust each input
    double bias;        // Baseline offset

    double compute(double[] inputs) {
        double sum = 0;
        for (int i = 0; i < inputs.length; i++) {
            sum += inputs[i] * weights[i]; // weighted sum
        }
        sum += bias;
        return relu(sum); // activation function (like a filter)
    }

    double relu(double x) { return Math.max(0, x); }
}
```

Step 2 — The Transformer Architecture (What makes LLMs special)

LLMs use **Transformers** — invented by Google in 2017 ('Attention is All You Need' paper).

Key innovation: **Self-Attention** — every word looks at every other word to understand context.

```
Sentence: 'The bank by the river was steep'

Without attention:  'bank' = financial institution? OR riverbank?
                    Model is confused!

With self-attention: 'bank' looks at 'river' → 'steep'
                     → 95% sure it means RIVERBANK ✅
```

Java Analogy for Attention:

```
// Self-Attention = a smart HashMap where every key
// queries every other key to understand relationships

public class SelfAttention {
    // Each word gets 3 vectors: Query, Key, Value
    // Q = 'What am I looking for?'
    // K = 'What do I contain?'
    // V = 'What do I share if relevant?'

    double[] computeAttention(Word[] words) {
        for (Word query : words) {
            for (Word key : words) {
                double score = dotProduct(query.Q, key.K);
                double weight = softmax(score);
                // weight close to 1 = 'very relevant!'
                // weight close to 0 = 'ignore this word'
```

```
            }
        }
    }
}
```

Step 3 — The Training Process

```
PHASE 1: DATA COLLECTION
━━━━━━━━━━━━━━━━━━━━━━━
Internet text + Books + Code + Wikipedia
→ ~15 TRILLION tokens (GPT-4 training data estimate)

PHASE 2: PRE-TRAINING (The expensive part)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
Task: 'Predict the next word'

Input:  'The cat sat on the ___'
Model predicts: 'mat' (30%), 'floor' (25%), 'chair' (20%)...
Actual answer:  'mat'

If WRONG → Calculate error (Loss) → Adjust weights (Backpropagation)
If RIGHT → Small adjustment, keep going

Repeat this 15 TRILLION times across 25,000 GPUs!

PHASE 3: FINE-TUNING (Make it helpful and safe)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
RLHF = Reinforcement Learning from Human Feedback

Human raters rank responses:
  Response A: 'Here's how to do it...' → ⭐⭐⭐⭐⭐
  Response B: 'I cannot help...'       → ⭐⭐
  Response C: (harmful response)       → ❌ REJECT

Model learns: 'Be helpful, be safe, follow instructions'
```

📡 Part 3: What Happens When You Send a Message?

The Complete Journey — From 'Enter' to Response

```
YOU TYPE: 'Write a Spring Boot REST controller for users'
Press ENTER...

STEP 1: Your Device (Browser/App)
  → HTTP/WebSocket request (HTTPS encrypted)

STEP 2: CDN / Load Balancer (Cloudflare / AWS CloudFront)
  → Routes to nearest data center
  → Handles DDoS protection + SSL termination

STEP 3: API Gateway / Auth
  → Validate API Key / Session Token
  → Rate limiting check
  → Route to correct service

STEP 4: Orchestration Service
  → Load conversation history
  → Build the full PROMPT:
    [SYSTEM]: You are a helpful assistant...
    [USER]:   Hi, what's Java?
    [ASST]:   Java is a programming language...
    [USER]:   Write a Spring Boot REST controller
  → Count tokens (within context window?)
  → Apply safety filters (pre-check)

STEP 5: Tokenization
```

```
  'Spring Boot REST' → [9435, 22841, 4717]
  (~1 token ≈ 0.75 words)

STEP 6: GPU Inference Cluster  ← THE EXPENSIVE PART
  H100 GPUs with model weights sharded across them

  FOR EACH TOKEN GENERATED:
  1. Embed tokens → high-dimensional vectors (e.g. 12288 dims)
  2. Pass through 96 Transformer layers
     Each layer:
     a. Self-Attention (every token looks at every other)
     b. Feed-Forward Network (process + transform)
     c. Layer Normalization
  3. Final layer → vocabulary logits (50,000+ possible tokens)
  4. Temperature/Top-P sampling → pick next token
  5. REPEAT until [END] token or max length reached

  ← Generates ONE token at a time (that's why it streams!)

STEP 7: Streaming / Detokenization
  Tokens → Words → sent back via SSE / WebSocket
  [1212][338][263]... → 'I' 'am' 'a'... (appears word by word!)

STEP 8: Post-Processing
  → Safety classifier check
  → Format response (markdown, code blocks)
  → Save to conversation history (DB)
  → Log for monitoring / billing

STEP 9: YOUR SCREEN 🖥️
  'Here's your Spring Boot controller...'
```

⏱️ Timing Breakdown (Typical request)

Event	Time
Network request to API	10–50ms
Auth + rate limiting	1–5ms
Prompt building + tokenization	5–20ms
Time to first token (TTFT)	200ms – 2s
Per-token generation time	10–50ms/token
Full response (500 tokens)	5–25 seconds
Network back to you	10–50ms

🔍 Token-by-Token Generation (Autoregressive)

```
Prompt: [Write][a][Spring][Boot][REST][controller][for][users]
                                                          ↓
                                          GPU processes ALL → predicts '@RestController'

New input: [...prompt...]['@RestController']
                                ↓
                          GPU → predicts '\n'

New input: [...]['@RestController']['\n']
                                ↓
                          GPU → predicts 'public'

...and so on until complete!
This is called AUTOREGRESSIVE generation.
```

🏗️ The Pipeline as Java/Spring Code

```
@RestController
public class ChatController {

    @PostMapping('/chat')
    public SseEmitter chat(@RequestBody ChatRequest req) {

        // 1. Auth
        authService.validate(req.apiKey);
        rateLimiter.check(req.userId);

        // 2. Build context
        List<Message> history = conversationRepo.findBySessionId(req.sessionId);
        String fullPrompt = promptBuilder.build(history, req.message);

        // 3. Tokenize
        int[] tokens = tokenizer.encode(fullPrompt);

        // 4. Inference (the magic black box)
        SseEmitter emitter = new SseEmitter();
        gpuCluster.generateStream(tokens, 0.7f)
            .subscribe(token → {
                String word = tokenizer.decode(token);
                emitter.send(word); // Stream to user!
            });

        // 5. Save history
        conversationRepo.save(req.sessionId, req.message, fullResponse);

        return emitter;
    }
}
```

🧱 Key Numbers to Remember

Thing	Scale
GPT-3 parameters	175 Billion
GPT-4 parameters (estimated)	~1.8 Trillion
Training data (GPT-4)	~13 Trillion tokens
Training cost (GPT-4)	~$100 Million
GPUs used for training	~25,000
Tokens per second (inference)	30–100 tokens/sec

TOPIC 3 — MCP (MODEL CONTEXT PROTOCOL)

YOU

Explain me with diagram what is MCP and how does it work?

ANTIGRAVITY AI

🔌 MCP — Model Context Protocol Explained with Diagrams

Think of MCP as the "USB standard for AI" — just like USB lets any device plug into any computer, MCP lets any AI connect to any tool/data source.

🤔 The Problem MCP Solves — 'Before MCP'

```
Before MCP (the chaos):
  ChatGPT  → custom code     → GitHub
  Claude   → different code  → GitHub
  Gemini   → yet another     → GitHub

Every AI × Every Tool = N × M integrations 😱
3 AIs × 10 tools = 30 custom integrations needed!

After MCP (the standard):
  ChatGPT ──┐
            ├──► [MCP] ──► GitHub
  Claude  ──┤         ├──► Filesystem
            │         └──► Database
  Gemini  ──┘

Any AI + Any Tool = N + M  (13 things, not 30!) ✅
```

📐 MCP Architecture — The Big Picture

```
┌──────────────────────────────────────────────────────┐
│         HOST APPLICATION (e.g., Claude Desktop)      │
│                                                      │
│   🤖 AI Model / LLM                                  │
│          ↕ Requests tools, resources, prompts        │
│   [MCP Client]                                       │
└──────────────────────┬───────────────────────────────┘
                       │ JSON-RPC 2.0 over stdio/HTTP/SSE
          ┌────────────┼────────────┐
          ↓            ↓            ↓
   [MCP Server]  [MCP Server]  [MCP Server]
   📁 Filesystem  🐙 GitHub    🗃️ Postgres
```

🏗️ MCP Building Blocks — 3 Things a Server Provides

Type	What It Is	Example
🔧 Tools	Functions AI can call	`read_file(path)`, `run_query(sql)`
📦 Resources	Data AI can read	`file://project/src/`, `db://users-table`
💬 Prompts	Reusable templates	`code review prompt`, `debug prompt`

🔄 How MCP Works — Step by Step

```
You: 'Read my pom.xml and find outdated dependencies'

1. AI thinks: 'I need to read a file. I have read_file tool!'
```

```
2. AI → MCP Client:
   call tool 'read_file' with args: {path: 'pom.xml'}

3. MCP Client → MCP Server (JSON-RPC):
   {
     'method': 'tools/call',
     'params': {
       'name': 'read_file',
       'arguments': {'path': 'pom.xml'}
     }
   }

4. MCP Server → reads actual file from filesystem

5. MCP Server → MCP Client:
   { 'result': { 'content': '<?xml ...dependencies...?>' } }

6. MCP Client → AI: 'Here's the file content: [pom.xml]'

7. AI reads XML, compares versions, generates response:
   'Found 3 outdated dependencies:
    - Spring Boot 2.7 → 3.2
    - Hibernate 5.6 → 6.4
    ...'
```

📡 MCP Communication Protocol — Under the Hood

```
MCP uses JSON-RPC 2.0 (like REST but for local communication)

STEP 1: INITIALIZATION (handshake)
Client → Server:
{
  'jsonrpc': '2.0',
  'method': 'initialize',
  'params': {
    'protocolVersion': '2024-11-05',
    'clientInfo': { 'name': 'Claude', 'version': '3.5' }
  }
}

Server → Client:
{
  'result': {
    'capabilities': {
      'tools': {},      ← 'I have tools you can call'
      'resources': {},  ← 'I have data you can read'
      'prompts': {}     ← 'I have prompt templates'
    }
  }
}

STEP 2: DISCOVER TOOLS
Client → Server: { 'method': 'tools/list' }

Server → Client:
{
  'tools': [
    {
      'name': 'read_file',
      'description': 'Read contents of a file',
      'inputSchema': { 'path': { 'type': 'string' } }
    },
    { 'name': 'write_file', 'description': '...' }
  ]
}

STEP 3: CALL A TOOL
```

```
Client → Server:
{ 'method': 'tools/call', 'params': { 'name': 'read_file', 'arguments': { 'path': '/project/pom.xml' } }

Server → Client:
{ 'result': { 'content': [{ 'type': 'text', 'text': '...' }] } }
```

🚀 Antigravity Uses MCP — Real Example

```
You: 'Fix the NullPointerException in UserService.java'

Antigravity AI engine:
  1. I decide: need to read file
     → MCP: read_file('UserService.java')
     ← Gets file contents

  2. I decide: grep for null checks
     → MCP: grep_search('pattern')
     ← Found issue at line 47

  3. I decide: write fixed code
     → MCP: write_file(fixed code)
     ← File written

  4. I decide: run tests
     → MCP: run_command('mvn test')
     ← Tests passed ✅

  5. Done! Fixed NPE at line 47.

MCP connects all these tools to me behind the scenes!
```

🛠️ Transport Methods

```
1. STDIO (most common for local tools)
   Client spawns Server as child process
   Communicates via stdin/stdout
   Example: Claude Desktop spawns 'node filesystem-server.js'

2. HTTP + SSE (for remote servers)
   Client connects to remote HTTP server
   Uses Server-Sent Events for streaming

3. WebSocket (bidirectional, real-time)
   Full duplex connection
   Great for real-time tool calling
```

☕ Building Your Own MCP Server in Java (Concept)

```
@McpServer(name = 'java-project-server')
public class JavaProjectMcpServer {

    // AI can call this function
    @McpTool(
        name = 'run_maven_build',
        description = 'Run Maven build and return results'
    )
    public ToolResult runMavenBuild(
        @McpParam('The Maven goal to run') String goal
    ) {
        ProcessResult result = execute('mvn ' + goal);
```

```
        return ToolResult.text(result.output());
    }

    // AI can check test results
    @McpTool(
        name = 'run_tests',
        description = 'Run JUnit tests and return pass/fail results'
    )
    public ToolResult runTests(
        @McpParam('Test class name') String testClass
    ) {
        ProcessResult result = execute('mvn test -Dtest=' + testClass);
        return ToolResult.text(result.output());
    }

    // AI can read project structure
    @McpResource(uri = 'project://structure')
    public ResourceContent getProjectStructure() {
        String tree = executeCommand('find . -name \
```

TOPIC 4 — CHAT EXPORT REQUEST

YOU

Can you export full chat to PDF?

ANTIGRAVITY AI

Sure! I've created this beautiful HTML export of our full conversation. Open it in your browser and press Cmd+P (Mac) or Ctrl+P (Windows) → Save as PDF to get a perfect PDF copy!

All 3 topics are included:

1. Gen AI, LLMs, Agents & Antigravity
2. GPU Architecture, LLM Training & Request Pipeline
3. MCP — Model Context Protocol

Generated by **Antigravity** — AI Coding Assistant by Google DeepMind

Conversation Date: May 23–26, 2026 | Topics: Gen AI, GPU/LLM Training, MCP Protocol